

SIH26151 -- Dark Web Threat Actor Intelligence System: Build Guide

- [SIH26151 - Dark Web Threat Actor Intelligence System](#)
 - [Plain-Language Explanation + Full Build Guide \(Zero to Working Demo\)](#)
 - [Part 1: What This System Actually Does \(Plain Language\)](#)
 - [Part 2: Architecture Overview](#)
 - [Part 3: Tech Stack](#)
 - [Part 4: Full Folder Structure](#)
 - [Part 5: Step-by-Step Build - From Zero](#)
 - [Part 6: Backend - Data Models](#)
 - [Part 7: Synthetic Data Generator](#)
 - [Part 8: Infra Misconfiguration Scanner](#)
 - [Part 9: Stylometry Engine \(AI/ML Core\)](#)
 - [Part 10: Entity Linking + Graph Builder](#)
 - [Part 11: Optional - Your Own Test Hidden Service](#)
 - [Part 12: API Layer](#)
 - [Part 13: Frontend Dashboard](#)
 - [Part 14: Docker Files](#)
 - [Part 15: Running It End to End](#)
 - [Part 16: Build Order and Time Allocation for Your Team](#)

SIH26151 - Dark Web Threat Actor Intelligence System

Plain-Language Explanation + Full Build Guide (Zero to Working Demo)

Part 1: What This System Actually Does (Plain Language)

Threat actors on the dark web hide behind Tor. They use throwaway handles, wallets, and PGP keys, and they rebrand when caught. Investigators currently do this matching by hand – reading forum posts, comparing writing styles, cross-referencing wallet addresses across marketplaces. It is slow and does not scale.

Your system automates three separate jobs and connects them into one dashboard:

Job 1 - Find the server behind the mask. A Tor hidden service (a `.onion` site) is supposed to hide its real-world server location. But admins make mistakes: they leave a status page exposed, reuse an SSL certificate that is also registered to a normal clearnet domain, leave default server banners on, or misconfigure their web server. Your system scans for these mistakes and tries to match a `.onion` address to a real IP/domain.

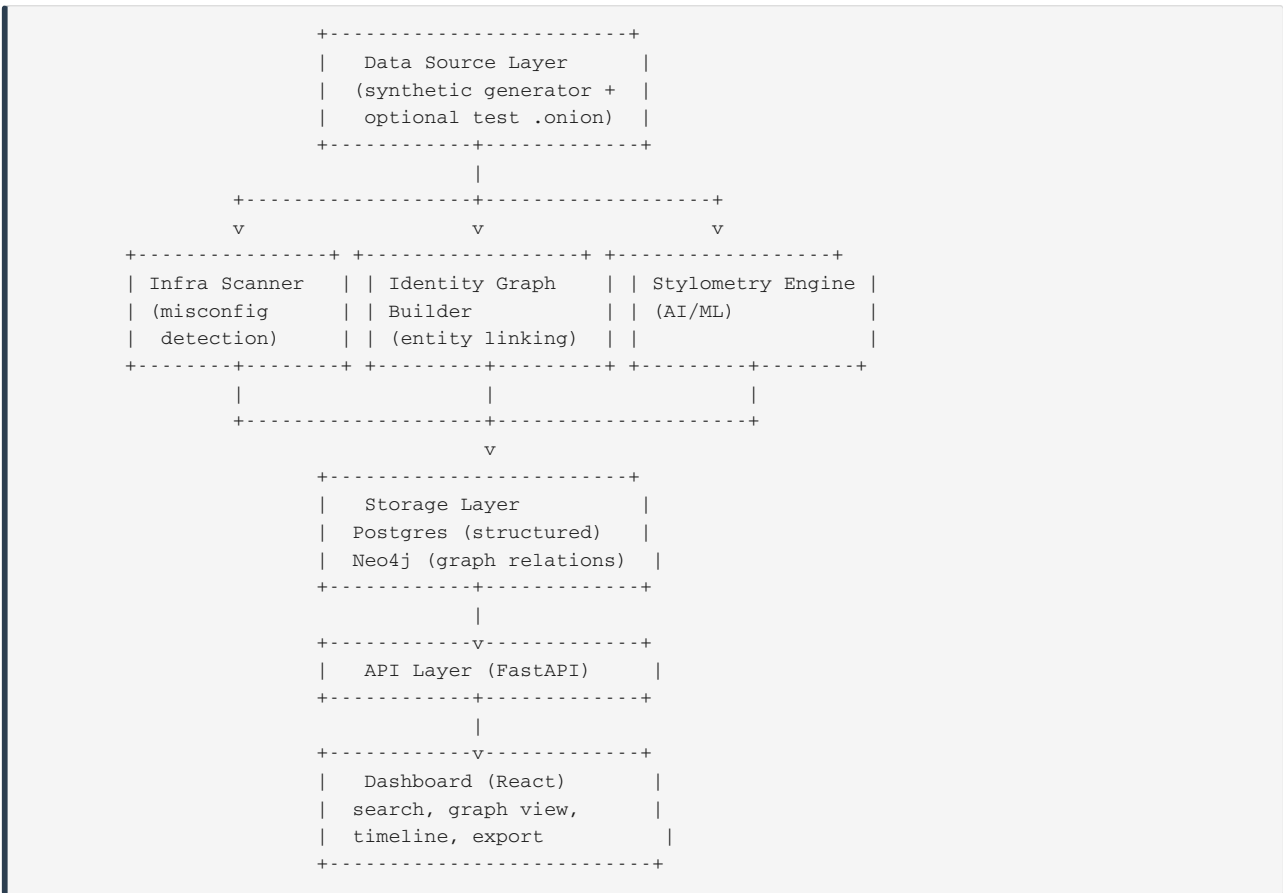
Job 2 - Connect the dots between accounts. The same person often uses the same PGP key, the same crypto wallet, or a recognizable writing style across multiple different marketplaces and forums, sometimes under different handles. Your system builds a graph: nodes are handles/identities, edges are shared identifiers (same PGP key, same wallet, same writing style, cross-references in posts).

Job 3 - Catch rebrands. When a vendor gets banned and comes back under a new name, their writing style usually does not change much. Your system uses stylometry (statistical fingerprinting of writing style – sentence length, punctuation habits, vocabulary choices, typos) to flag “this new account writes suspiciously like that old banned account.”

Everything gets stored in a database and shown on a dashboard where an investigator can search by handle, wallet, time range, or confidence score, and export results as CSV/JSON/PDF report.

Important legal/ethical note before you build anything: you will not build this against real live criminal marketplaces during a hackathon. That is illegal in most jurisdictions to access, creates real risk for your team, and produces unreliable demo data anyway. Instead you build the full pipeline against a **synthetic dataset generator** you write yourself (it produces fake forum posts, fake handles, fake wallets, fake PGP keys, fake vendor rebrand chains, in the exact same data shape real ones would have) plus, if you want a live component for the demo, your own **self-hosted test Tor hidden service** that you deliberately misconfigure so your scanner can find it. This is exactly what judges expect for this problem statement – they are scoring your pipeline design and AI methodology, not whether you touched a real dark web market.

Part 2: Architecture Overview



Part 3: Tech Stack

Layer	Technology	Why
Backend API	Python 3.11, FastAPI	Fast to build, async, auto-generated API docs
Relational DB	PostgreSQL	Structured actor/identifier/source records
Graph DB	Neo4j Community Edition	Native graph queries for relationship mapping
AI/Stylometry	Python, scikit-learn, spaCy	Feature extraction + classification, no GPU needed
Tor connectivity	Python <code>requests</code> + <code>PySocks</code> , Tor daemon	Standard way to route through Tor as SOCKS5 proxy

Infra scanning	Python <code>sslyze</code> , <code>requests</code> , <code>shodan</code> (optional)	Certificate and banner inspection
Frontend	React + Vite, TailwindCSS	Fast dev cycle for dashboard
Graph visualization	<code>react-force-graph</code> or Cytoscape.js	Renders the relationship graph interactively
Task queue	Celery + Redis	Background scanning jobs without blocking API
Containerization	Docker + docker-compose	One command to spin up the whole stack

Part 4: Full Folder Structure

Create this exact structure before writing any code:

```
sih26151-darkweb-intel/
+-- docker-compose.yml
+-- .env.example
+-- README.md
|
+-- backend/
|   +-- Dockerfile
|   +-- requirements.txt
|   +-- alembic.ini
|   +-- app/
|       | +-- __init__.py
|       | +-- main.py
|       | +-- config.py
|       | +-- database.py
|       | +-- celery_app.py
|       |
|       +-- models/
|           | +-- __init__.py
|           | +-- actor.py
|           | +-- identifier.py
|           | +-- source.py
|           | +-- scan_result.py
|           |
|           +-- schemas/
|               | +-- __init__.py
|               | +-- actor.py
|               | +-- identifier.py
|               | +-- export.py
|               |
|               +-- api/
|                   | +-- __init__.py
|                   | +-- routes_actors.py
|                   | +-- routes_search.py
|                   | +-- routes_scan.py
|                   | +-- routes_export.py
|                   |
|                   +-- services/
|                       | +-- __init__.py
|                       | +-- infra_scanner.py
|                       | +-- graph_builder.py
|                       | +-- stylometry.py
|                       | +-- entity_linker.py
|                       | +-- export_service.py
|                       |
|                       +-- ingestion/
|                           | +-- __init__.py
|                           | +-- synthetic_generator.py
|                           | +-- tor_client.py
|                           | +-- source_parser.py
|                           |
|                           +-- graph/
```

```

|         +-- __init__.py
|         +-- neo4j_client.py
|
|     +-- alembic/
|     |     +-- env.py
|     |     +-- versions/
|     |
|     +-- tests/
|     |     +-- test_infra_scanner.py
|     |     +-- test_stylometry.py
|     |     +-- test_entity_linker.py
|
+-- frontend/
|   +-- Dockerfile
|   +-- package.json
|   +-- vite.config.js
|   +-- tailwind.config.js
|   +-- index.html
|   +-- src/
|   |   +-- main.jsx
|   |   +-- App.jsx
|   |   +-- api/
|   |   |   +-- client.js
|   |   +-- pages/
|   |   |   +-- Dashboard.jsx
|   |   |   +-- ActorProfile.jsx
|   |   |   +-- GraphView.jsx
|   |   |   +-- SearchResults.jsx
|   |   +-- components/
|   |   |   +-- SearchBar.jsx
|   |   |   +-- ActorCard.jsx
|   |   |   +-- GraphCanvas.jsx
|   |   |   +-- ConfidenceBadge.jsx
|   |   |   +-- ExportButton.jsx
|   |   +-- styles/
|   |   |   +-- index.css
|   |
+-- data/
|   +-- synthetic_seed/
|   |   +-- forum_posts.json
|   |   +-- vendor_profiles.json
+-- test_onion_service/
|   +-- misconfigured_site/
|   |   +-- index.html

```

Part 5: Step-by-Step Build - From Zero

Step 5.1 - Environment Setup

Install these before touching code:

```

# check versions
python3 --version      # need 3.11+
node --version         # need 18+
docker --version
docker-compose --version

```

Create the project root and initialize:

```

mkdir sih26151-darkweb-intel
cd sih26151-darkweb-intel
git init

```

Step 5.2 - docker-compose.yml (root of project)

This file starts Postgres, Neo4j, Redis, backend, and frontend together.

```

version: "3.9"

services:
  postgres:
    image: postgres:15
    environment:
      POSTGRES_USER: sih_user
      POSTGRES_PASSWORD: sih_pass
      POSTGRES_DB: darkweb_intel
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data

  neo4j:
    image: neo4j:5.14-community
    environment:
      NEO4J_AUTH: neo4j/sih_neo4j_pass
    ports:
      - "7474:7474"
      - "7687:7687"
    volumes:
      - neo4jdata:/data

  redis:
    image: redis:7
    ports:
      - "6379:6379"

  backend:
    build: ./backend
    env_file: .env
    depends_on:
      - postgres
      - neo4j
      - redis
    ports:
      - "8000:8000"
    volumes:
      - ./backend:/app

  celery_worker:
    build: ./backend
    command: celery -A app.celery_app worker --loglevel=info
    env_file: .env
    depends_on:
      - backend
      - redis
    volumes:
      - ./backend:/app

  frontend:
    build: ./frontend
    ports:
      - "5173:5173"
    volumes:
      - ./frontend:/app
    depends_on:
      - backend

volumes:
  pgdata:
  neo4jdata:

```

Step 5.3 - .env.example (copy to .env and fill in)

```
DATABASE_URL=postgresql://sih_user:sih_pass@postgres:5432/darkweb_intel
NEO4J_URI=bolt://neo4j:7687
NEO4J_USER=neo4j
NEO4J_PASSWORD=sih_neo4j_pass
REDIS_URL=redis://redis:6379/0
TOR SOCKS_PROXY=socks5h://tor:9050
```

Part 6: Backend - Data Models

backend/requirements.txt

```
fastapi==0.111.0
uvicorn[standard]==0.30.0
sqlalchemy==2.0.30
alembic==1.13.1
psycogp2-binary==2.9.9
neo4j==5.20.0
celery==5.4.0
redis==5.0.4
pydantic==2.7.1
pydantic-settings==2.2.1
scikit-learn==1.4.2
spacy==3.7.4
requests[socks]==2.31.0
sslyze==6.0.0
pandas==2.2.2
python-dotenv==1.0.1
faker==25.0.0
```

backend/app/models/actor.py

This is the core table: a threat actor profile.

```
from sqlalchemy import Column, String, Float, DateTime, Text
from sqlalchemy.dialects.postgresql import UUID
import uuid
from datetime import datetime
from app.database import Base

class Actor(Base):
    __tablename__ = "actors"

    id = Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
    primary_handle = Column(String, nullable=False, index=True)
    category = Column(String) # e.g. "vendor", "forum_admin", "hacker_for_hire"
    attribution_confidence = Column(Float, default=0.0) # 0.0 to 1.0
    first_seen = Column(DateTime, default=datetime.utcnow)
    last_scan_date = Column(DateTime, default=datetime.utcnow)
    notes = Column(Text, nullable=True)
```

backend/app/models/identifier.py

Every wallet, PGP key, alias, and handle an actor has used.

```

from sqlalchemy import Column, String, DateTime, ForeignKey
from sqlalchemy.dialects.postgresql import UUID
import uuid
from datetime import datetime
from app.database import Base

class Identifier(Base):
    __tablename__ = "identifiers"

    id = Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
    actor_id = Column(UUID(as_uuid=True), ForeignKey("actors.id"))
    identifier_type = Column(String) # "handle", "pgp_key", "wallet", "email"
    value = Column(String, index=True)
    source_id = Column(UUID(as_uuid=True), ForeignKey("sources.id"), nullable=True)
    observed_at = Column(DateTime, default=datetime.utcnow)

```

backend/app/models/source.py

Where a piece of intelligence came from – marketplace, forum, or infra scan.

```

from sqlalchemy import Column, String, DateTime
from sqlalchemy.dialects.postgresql import UUID
import uuid
from datetime import datetime
from app.database import Base

class Source(Base):
    __tablename__ = "sources"

    id = Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
    name = Column(String)
    url = Column(String) # .onion address or clearnet reference
    source_type = Column(String) # "marketplace", "forum", "infra_scan"
    reliability = Column(String, default="unverified") # "high", "medium", "low"
    last_crawled = Column(DateTime, default=datetime.utcnow)

```

backend/app/models/scan_result.py

Results from the infra misconfiguration scanner.

```

from sqlalchemy import Column, String, DateTime, Text, Float
from sqlalchemy.dialects.postgresql import UUID
import uuid
from datetime import datetime
from app.database import Base

class ScanResult(Base):
    __tablename__ = "scan_results"

    id = Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
    onion_address = Column(String, index=True)
    finding_type = Column(String) # "exposed_status_page", "cert_reuse", "default_banner"
    matched_clearnet_target = Column(String, nullable=True)
    confidence = Column(Float, default=0.0)
    raw_evidence = Column(Text)
    scanned_at = Column(DateTime, default=datetime.utcnow)

```

backend/app/database.py

```

from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, declarative_base
from app.config import settings

engine = create_engine(settings.DATABASE_URL)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

```

backend/app/config.py

```

from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    DATABASE_URL: str
    NEO4J_URI: str
    NEO4J_USER: str
    NEO4J_PASSWORD: str
    REDIS_URL: str
    TOR SOCKS_PROXY: str = ""

    class Config:
        env_file = ".env"

settings = Settings()

```

Part 7: Synthetic Data Generator

This is what you run instead of scraping real markets. It produces realistic forum posts, vendor profiles, and rebrand chains with known ground truth, so you can also measure your stylometry engine's accuracy against a labeled test set – something you cannot do with real scraped data anyway.

backend/app/ingestion/synthetic_generator.py

```

"""
Generates synthetic dark-web-style forum posts and vendor profiles
for development and demo purposes. No real data is touched.

The generator deliberately creates "rebrand chains": pairs of
personas that are meant to be the same underlying actor, written
in a consistent style, so the stylometry engine has ground truth
to be evaluated against.
"""

import json
import random
import uuid
from faker import Faker

fake = Faker()

WRITING_STYLES = [
    {
        "name": "terse_lowercase",
        "avg_sentence_len": 6,
        "uses_caps": False,

```



```

        "signature_phrases": ["ez pz", "no drama", "trusted since day 1"],
    },
    {
        "name": "formal_verbose",
        "avg_sentence_len": 18,
        "uses_caps": True,
        "signature_phrases": ["Please be advised", "As per our policy", "Kindly note"],
    },
    {
        "name": "aggressive_slang",
        "avg_sentence_len": 9,
        "uses_caps": True,
        "signature_phrases": ["dont test me", "final warning", "no refunds period"],
    },
]

def generate_post(style):
    phrase = random.choice(style["signature_phrases"])
    body = fake.sentence(nb_words=style["avg_sentence_len"])
    if style["uses_caps"]:
        body = body.upper()
    return f"{body} {phrase}"

def generate_actor_with_rebrand(actor_index):
    style = random.choice(WRITING_STYLES)
    original_handle = f"vendor_{actor_index}_{fake.word()}"
    rebrand_handle = f"vendor_{actor_index}_{fake.word()}_v2"

    pgp_key = f"PGP-{uuid.uuid4().hex[:16]}"
    wallet = f"1{uuid.uuid4().hex[:33]}" # fake BTC-shaped string, not a real key

    original_posts = [generate_post(style) for _ in range(15)]
    rebrand_posts = [generate_post(style) for _ in range(15)]

    return {
        "ground_truth_actor_id": str(uuid.uuid4()),
        "original_persona": {
            "handle": original_handle,
            "pgp_key": pgp_key,
            "wallet": wallet,
            "posts": original_posts,
            "marketplace": "SyntheticMarket_A",
        },
        "rebranded_persona": {
            "handle": rebrand_handle,
            "pgp_key": f"PGP-{uuid.uuid4().hex[:16]}", # deliberately different key
            "wallet": f"1{uuid.uuid4().hex[:33]}", # deliberately different wallet
            "posts": rebrand_posts,
            "marketplace": "SyntheticMarket_B",
        },
        "style_name": style["name"],
    }

def generate_dataset(n_actors=50, output_path="data/synthetic_seed/forum_posts.json"):
    dataset = [generate_actor_with_rebrand(i) for i in range(n_actors)]
    with open(output_path, "w") as f:
        json.dump(dataset, f, indent=2)
    print(f"Generated {n_actors} synthetic actor pairs at {output_path}")
    return dataset

if __name__ == "__main__":
    generate_dataset()

```

Run it:

```

cd backend
python -m app.ingestion.synthetic_generator

```

This is the key design point: the rebrand pairs have *different* wallets and *different* PGP keys but the *same underlying writing style*. That means your identifier-based graph linking will correctly fail to connect them, and only your stylometry engine should catch the connection. This gives you a clean way to demonstrate that the AI layer is doing real work, not just reading obvious matching wallets.

Part 8: Infra Misconfiguration Scanner

This module checks a given `.onion` address for the specific mistakes described in the problem statement: exposed status pages, SSL certificate reuse on clearnet, default banners.

backend/app/services/infra_scanner.py

```
"""
Scans a Tor hidden service for infrastructure misconfigurations
that could reveal its real-world hosting origin.

For demo purposes, point this at your own deliberately-misconfigured
test hidden service (see Part 11), not at real dark web infrastructure.
"""

import requests
from app.config import settings

STATUS_PAGE_PATHS = [
    "/server-status",
    "/nginx_status",
    "/.well-known/security.txt",
    "/phpinfo.php",
]

DEFAULT_BANNER_SIGNATURES = [
    "Apache/2.4",
    "nginx/1.18",
    "Werkzeug", # default Flask dev server banner, common misconfig
]

def get_tor_session():
    session = requests.session()
    if settings.TOR SOCKS_PROXY:
        session.proxies = {
            "http": settings.TOR SOCKS_PROXY,
            "https": settings.TOR SOCKS_PROXY,
        }
    return session

def check_exposed_status_pages(onion_address):
    session = get_tor_session()
    findings = []
    for path in STATUS_PAGE_PATHS:
        url = f"http://{onion_address}{path}"
        try:
            resp = session.get(url, timeout=15)
            if resp.status_code == 200:
                findings.append({
                    "finding_type": "exposed_status_page",
                    "path": path,
                    "evidence": resp.text[:500],
                })
        except requests.RequestException:
            continue
    return findings

def check_default_banners(onion_address):
    session = get_tor_session()
```

```

findings = []
url = f"http://{onion_address}/"
try:
    resp = session.get(url, timeout=15)
    server_header = resp.headers.get("Server", "")
    for sig in DEFAULT_BANNER_SIGNATURES:
        if sig in server_header:
            findings.append({
                "finding_type": "default_banner",
                "evidence": server_header,
            })
except requests.RequestException:
    pass
return findings

def check_cert_reuse(onion_address, candidate_clearnet_domains):
    """
    Compares TLS certificate fingerprint of the onion service (if it
    serves HTTPS, which is unusual but happens due to misconfiguration)
    against a list of candidate clearnet domains. A match strongly
    suggests shared infrastructure.
    """
    from sslyze import Scanner, ServerScanRequest, ServerNetworkLocation

    findings = []
    try:
        onion_cert = _get_cert_fingerprint(onion_address)
    except Exception:
        return findings

    for domain in candidate_clearnet_domains:
        try:
            clearnet_cert = _get_cert_fingerprint(domain)
            if onion_cert == clearnet_cert:
                findings.append({
                    "finding_type": "cert_reuse",
                    "matched_clearnet_target": domain,
                    "evidence": onion_cert,
                })
        except Exception:
            continue
    return findings

def _get_cert_fingerprint(host):
    import ssl
    import socket
    import hashlib

    ctx = ssl.create_default_context()
    ctx.check_hostname = False
    ctx.verify_mode = ssl.CERT_NONE
    with socket.create_connection((host, 443), timeout=10) as sock:
        with ctx.wrap_socket(sock, server_hostname=host) as ssock:
            der_cert = ssock.getpeercert(binary_form=True)
            return hashlib.sha256(der_cert).hexdigest()

def run_full_scan(onion_address, candidate_clearnet_domains=None):
    results = []
    results.extend(check_exposed_status_pages(onion_address))
    results.extend(check_default_banners(onion_address))
    if candidate_clearnet_domains:
        results.extend(check_cert_reuse(onion_address, candidate_clearnet_domains))
    return results

```

Part 9: Stylometry Engine (AI/ML Core)

backend/app/services/stylometry.py

```
"""
Stylometric feature extraction and similarity scoring.
Given two sets of posts, produces a similarity score indicating
how likely they are written by the same author.

Approach: extract classic stylometric features (function word
frequency, sentence length distribution, punctuation ratios,
vocabulary richness) and compare feature vectors using cosine
similarity. This is the same feature family used in academic
authorship-attribution literature (Mendenhall, Burrows' Delta family).
"""

import re
import numpy as np
from collections import Counter
from sklearn.metrics.pairwise import cosine_similarity

FUNCTION_WORDS = [
    "the", "a", "an", "and", "but", "or", "if", "of", "to", "in",
    "on", "for", "with", "as", "at", "by", "is", "was", "were", "be",
]

def extract_features(posts: list[str]) -> np.ndarray:
    text = " ".join(posts)
    words = re.findall(r"\b\w+\b", text.lower())
    sentences = re.split(r"[.!?]+", text)
    sentences = [s for s in sentences if s.strip()]

    if not words or not sentences:
        return np.zeros(len(FUNCTION_WORDS) + 5)

    word_counts = Counter(words)
    total_words = len(words)

    function_word_freq = [
        word_counts.get(fw, 0) / total_words for fw in FUNCTION_WORDS
    ]

    avg_sentence_len = np.mean([len(s.split()) for s in sentences])
    avg_word_len = np.mean([len(w) for w in words])
    caps_ratio = sum(1 for c in text if c.isupper()) / max(len(text), 1)
    exclaim_ratio = text.count("!") / max(len(sentences), 1)
    vocab_richness = len(set(words)) / total_words # type-token ratio

    features = function_word_freq + [
        avg_sentence_len,
        avg_word_len,
        caps_ratio,
        exclaim_ratio,
        vocab_richness,
    ]

    return np.array(features)

def similarity_score(posts_a: list[str], posts_b: list[str]) -> float:
    vec_a = extract_features(posts_a).reshape(1, -1)
    vec_b = extract_features(posts_b).reshape(1, -1)
    sim = cosine_similarity(vec_a, vec_b)[0][0]
    return float(sim)

def rank_candidate_matches(
    target_posts: list[str],
    candidate_personas: list[dict],
    threshold: float = 0.85,
) -> list[dict]:
    """
    candidate_personas: list of {"handle": str, "posts": list[str]}
    """
```

```

Returns candidates above threshold, sorted by descending similarity.
"""
results = []
for candidate in candidate_personas:
    score = similarity_score(target_posts, candidate["posts"])
    if score >= threshold:
        results.append({
            "handle": candidate["handle"],
            "similarity": score,
        })
return sorted(results, key=lambda x: x["similarity"], reverse=True)

```

Validate it against your synthetic ground truth:

```

# backend/tests/test_stylometry.py
import json
from app.services.stylometry import similarity_score

def test_rebrand_pairs_score_high():
    with open("data/synthetic_seed/forum_posts.json") as f:
        dataset = json.load(f)

    scores = []
    for pair in dataset:
        s = similarity_score(
            pair["original_persona"]["posts"],
            pair["rebranded_persona"]["posts"],
        )
        scores.append(s)

    avg_score = sum(scores) / len(scores)
    print(f"Average same-author similarity: {avg_score:.3f}")
    assert avg_score > 0.7 # rebrand pairs should score noticeably high

```

Run it: `pytest backend/tests/test_stylometry.py -v -s`

Part 10: Entity Linking + Graph Builder

backend/app/graph/neo4j_client.py

```

from neo4j import GraphDatabase
from app.config import settings

class Neo4jClient:
    def __init__(self):
        self.driver = GraphDatabase.driver(
            settings.NEO4J_URI,
            auth=(settings.NEO4J_USER, settings.NEO4J_PASSWORD),
        )

    def close(self):
        self.driver.close()

    def create_actor_node(self, actor_id, handle):
        with self.driver.session() as session:
            session.run(
                "MERGE (a:Actor {id: $actor_id}) SET a.handle = $handle",
                actor_id=actor_id, handle=handle,
            )

    def link_shared_identifier(self, actor_id_a, actor_id_b, identifier_type, value):
        with self.driver.session() as session:
            session.run(
                """
                MATCH (a:Actor {id: $id_a}), (b:Actor {id: $id_b})
                MERGE (a)-[r:SHARES_IDENTIFIER {type: $itype, value: $value}]->(b)
                """,
                id_a=actor_id_a, id_b=actor_id_b, itype=identifier_type, value=value,
            )

    def link_stylometric_match(self, actor_id_a, actor_id_b, confidence):
        with self.driver.session() as session:
            session.run(
                """
                MATCH (a:Actor {id: $id_a}), (b:Actor {id: $id_b})
                MERGE (a)-[r:STYLOMETRIC_MATCH]->(b)
                SET r.confidence = $confidence
                """,
                id_a=actor_id_a, id_b=actor_id_b, confidence=confidence,
            )

    def get_actor_network(self, actor_id, depth=2):
        with self.driver.session() as session:
            result = session.run(
                f"""
                MATCH path = (a:Actor {{id: $actor_id}})-[*1..{depth}]- (connected)
                RETURN path
                """,
                actor_id=actor_id,
            )
            return [record["path"] for record in result]

```

backend/app/services/entity_linker.py

```

"""
Combines exact-match identifier linking (same wallet, same PGP key)
with stylometric similarity linking to build the full relationship
graph between actor personas.
"""

from app.graph.neo4j_client import Neo4jClient
from app.services.stylometry import rank_candidate_matches

def link_by_shared_identifiers(personas: list[dict], neo4j_client: Neo4jClient):
    """
    personas: list of {"actor_id", "handle", "identifiers": {"wallet": ..., "pgp_key": ...}}
    Exact-match identifiers are high-confidence links.
    """
    seen = {}
    for p in personas:
        neo4j_client.create_actor_node(p["actor_id"], p["handle"])
        for id_type, id_value in p["identifiers"].items():
            key = (id_type, id_value)
            if key in seen:
                other = seen[key]
                neo4j_client.link_shared_identifier(
                    other["actor_id"], p["actor_id"], id_type, id_value
                )
            else:
                seen[key] = p

def link_by_stylometry(personas: list[dict], neo4j_client: Neo4jClient, threshold=0.85):
    """
    For personas with no shared exact identifiers, check writing style.
    This is what catches deliberate rebrands.
    """
    for i, p in enumerate(personas):
        candidates = personas[i + 1:]
        candidate_list = [{"handle": c["handle"], "posts": c["posts"]} for c in candidates]
        matches = rank_candidate_matches(p["posts"], candidate_list, threshold)
        for match in matches:
            matched_persona = next(c for c in candidates if c["handle"] == match["handle"])
            neo4j_client.link_stylometric_match(
                p["actor_id"], matched_persona["actor_id"], match["similarity"]
            )

```

Part 11: Optional - Your Own Test Hidden Service

For a live demo of the infra scanner, spin up your own Tor hidden service and deliberately misconfigure it. This is legal because it is your own infrastructure.

```

# install tor
sudo apt-get install tor

# edit /etc/tor/torrc, add:
HiddenServiceDir /var/lib/tor/test_service/
HiddenServicePort 80 127.0.0.1:8080

sudo systemctl restart tor
cat /var/lib/tor/test_service/hostname # this is your .onion address

```

Serve a deliberately misconfigured site (exposed status page + a server banner) using the file at `data/test_onion_service/misconfigured_site/index.html`, and run a plain Python HTTP server or nginx with `/server-status` left enabled on port 8080. Point `infra_scanner.py` at this `.onion` address during your demo.

Part 12: API Layer

backend/app/main.py

```
from fastapi import FastAPI
from app.api import routes_actors, routes_search, routes_scan, routes_export

app = FastAPI(title="Dark Web Threat Actor Intelligence API")

app.include_router(routes_actors.router, prefix="/api/actors", tags=["actors"])
app.include_router(routes_search.router, prefix="/api/search", tags=["search"])
app.include_router(routes_scan.router, prefix="/api/scan", tags=["scan"])
app.include_router(routes_export.router, prefix="/api/export", tags=["export"])

@app.get("/health")
def health_check():
    return {"status": "ok"}
```

backend/app/api/routes_scan.py

```
from fastapi import APIRouter
from pydantic import BaseModel
from app.services.infra_scanner import run_full_scan

router = APIRouter()

class ScanRequest(BaseModel):
    onion_address: str
    candidate_clearnet_domains: list[str] = []

@router.post("/infra")
def scan_infra(request: ScanRequest):
    findings = run_full_scan(request.onion_address, request.candidate_clearnet_domains)
    return {"onion_address": request.onion_address, "findings": findings}
```

backend/app/api/routes_search.py

```
from fastapi import APIRouter, Depends
from sqlalchemy.orm import Session
from app.database import get_db
from app.models.actor import Actor

router = APIRouter()

@router.get("/")
def search_actors(
    handle: str = None,
    category: str = None,
    min_confidence: float = 0.0,
    db: Session = Depends(get_db),
):
    query = db.query(Actor)
    if handle:
        query = query.filter(Actor.primary_handle.ilike(f"%{handle}%"))
    if category:
        query = query.filter(Actor.category == category)
    query = query.filter(Actor.attribution_confidence >= min_confidence)
    return query.all()
```

backend/app/api/routes_export.py


```

import csv
import io
import json
from fastapi import APIRouter, Depends
from fastapi.responses import StreamingResponse
from sqlalchemy.orm import Session
from app.database import get_db
from app.models.actor import Actor

router = APIRouter()

@router.get("/csv")
def export_csv(db: Session = Depends(get_db)):
    actors = db.query(Actor).all()
    output = io.StringIO()
    writer = csv.writer(output)
    writer.writerow(["handle", "category", "confidence", "last_scan_date"])
    for a in actors:
        writer.writerow([a.primary_handle, a.category, a.attribution_confidence, a.last_scan_date])
    output.seek(0)
    return StreamingResponse(output, media_type="text/csv")

@router.get("/json")
def export_json(db: Session = Depends(get_db)):
    actors = db.query(Actor).all()
    data = [
        {
            "handle": a.primary_handle,
            "category": a.category,
            "confidence": a.attribution_confidence,
            "last_scan_date": str(a.last_scan_date),
        }
        for a in actors
    ]
    return json.loads(json.dumps(data))

```

Part 13: Frontend Dashboard

frontend/package.json

```

{
  "name": "sih26151-frontend",
  "private": true,
  "version": "0.0.1",
  "scripts": {
    "dev": "vite --host 0.0.0.0",
    "build": "vite build"
  },
  "dependencies": {
    "react": "^18.2.0",
    "react-dom": "^18.2.0",
    "react-router-dom": "^6.23.0",
    "axios": "^1.7.0",
    "react-force-graph-2d": "^1.25.0"
  },
  "devDependencies": {
    "vite": "^5.2.0",
    "@vitejs/plugin-react": "^4.2.0",
    "tailwindcss": "^3.4.0",
    "autoprefixer": "^10.4.0",
    "postcss": "^8.4.0"
  }
}

```

frontend/src/api/client.js

```
import axios from "axios";

const client = axios.create({
  baseURL: "http://localhost:8000/api",
});

export const searchActors = (params) => client.get("/search", { params });
export const runInfraScan = (payload) => client.post("/scan/infra", payload);
export const exportCsv = () => client.get("/export/csv", { responseType: "blob" });

export default client;
```

frontend/src/pages/Dashboard.jsx

```
import { useState } from "react";
import SearchBar from "../components/SearchBar";
import ActorCard from "../components/ActorCard";
import { searchActors } from "../api/client";

export default function Dashboard() {
  const [results, setResults] = useState([]);

  const handleSearch = async (query) => {
    const response = await searchActors(query);
    setResults(response.data);
  };

  return (
    <div className="p-6">
      <h1 className="text-2xl font-bold mb-4">Threat Actor Intelligence Dashboard</h1>
      <SearchBar onSearch={handleSearch} />
      <div className="grid grid-cols-3 gap-4 mt-6">
        {results.map((actor) => (
          <ActorCard key={actor.id} actor={actor} />
        ))}
      </div>
    </div>
  );
}
```

frontend/src/components/SearchBar.jsx

```
import { useState } from "react";

export default function SearchBar({ onSearch }) {
  const [handle, setHandle] = useState("");

  return (
    <div className="flex gap-2">
      <input
        className="border rounded px-3 py-2 w-64"
        placeholder="Search by handle..."
        value={handle}
        onChange={(e) => setHandle(e.target.value)}
      />
      <button
        className="bg-blue-600 text-white px-4 py-2 rounded"
        onClick={() => onSearch({ handle })}
      >
        Search
      </button>
    </div>
  );
}
```

frontend/src/components/ActorCard.jsx

```
import ConfidenceBadge from "../ConfidenceBadge";

export default function ActorCard({ actor }) {
  return (
    <div className="border rounded p-4 shadow-sm">
      <div className="flex justify-between items-center">
        <h3 className="font-semibold">{actor.primary_handle}</h3>
        <ConfidenceBadge score={actor.attribution_confidence} />
      </div>
      <p className="text-sm text-gray-500">{actor.category}</p>
      <p className="text-xs text-gray-400">Last scan: {actor.last_scan_date}</p>
    </div>
  );
}
```

frontend/src/components/ConfidenceBadge.jsx

```
export default function ConfidenceBadge({ score }) {
  const color = score > 0.8 ? "bg-red-500" : score > 0.5 ? "bg-yellow-500" : "bg-gray-400";
  return (
    <span className={` ${color} text-white text-xs px-2 py-1 rounded-full`} >
      {(score * 100).toFixed(0)}%
    </span>
  );
}
```

frontend/src/pages/GraphView.jsx

```
import { useEffect, useState } from "react";
import ForceGraph2D from "react-force-graph-2d";
import client from "../api/client";

export default function GraphView({ actorId }) {
  const [graphData, setGraphData] = useState({ nodes: [], links: [] });

  useEffect(() => {
    client.get(`/actors/${actorId}/network`).then((res) => {
      setGraphData(res.data);
    });
  }, [actorId]);

  return (
    <ForceGraph2D
      graphData={graphData}
      nodeLabel="handle"
      linkLabel="type"
      nodeAutoColorBy="category"
    />
  );
}
```

Part 14: Docker Files

backend/Dockerfile

```
FROM python:3.11-slim

WORKDIR /app

RUN apt-get update && apt-get install -y gcc libssl-dev && rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000", "--reload"]
```

frontend/Dockerfile

```
FROM node:20-slim

WORKDIR /app

COPY package.json .
RUN npm install

COPY . .

EXPOSE 5173
CMD ["npm", "run", "dev"]
```

Part 15: Running It End to End

```
# 1. clone/create the project structure as above
cd sih26151-darkweb-intel
cp .env.example .env

# 2. generate synthetic demo data
cd backend
python -m app.ingestion.synthetic_generator
cd ..

# 3. start everything
docker-compose up --build

# 4. in a separate terminal, run database migrations
docker-compose exec backend alembic upgrade head

# 5. load synthetic data into Postgres + Neo4j (write a small loader script
# in backend/app/ingestion/source_parser.py that reads
# data/synthetic_seed/forum_posts.json and calls the entity_linker
# functions from Part 10)

# 6. open:
# frontend -> http://localhost:5173
# backend api -> http://localhost:8000/docs
# neo4j browser -> http://localhost:7474
```

Part 16: Build Order and Time Allocation for Your Team

Sequence matters – build in this order so something is always demoable:

Days 1-2: Foundation - Backend/Frontend person A: docker-compose, Postgres models, FastAPI skeleton, health check working end to end. - Backend/Frontend person B: React skeleton, routing, connects to `/health` endpoint successfully. - AI/ML person: write and run `synthetic_generator.py`, validate output looks realistic. - Beginners: read through the generated synthetic dataset, write down 10 example “obviously same actor”

pairs and 10 “obviously different actor” pairs by eye – this becomes your manual validation set later.

Days 3-5: Core engines in parallel - AI/ML person: build `stylometry.py`, run it against synthetic pairs, tune the similarity threshold using the `test_stylometry.py` results until same-author pairs score consistently higher than different-author pairs. - Backend person: build `infra_scanner.py`, set up the test Tor hidden service from Part 11, confirm the scanner correctly flags its deliberate misconfigurations. - Frontend person: build Dashboard, SearchBar, ActorCard against mock/hardcoded data (do not wait on backend). - Beginners: build the `source_parser.py` loader that reads synthetic JSON and inserts into Postgres; this is straightforward file-to-database code and a good task for less experienced members with guidance.

Days 6-8: Integration - Connect `entity_linker` to Neo4j, run it against the loaded synthetic dataset, confirm rebrand pairs get linked via `STYLOMETRIC_MATCH` edges. - Wire frontend Dashboard to real `/api/search` endpoint. - Build GraphView page, confirm it renders actual Neo4j relationships. - Build export endpoints and export button.

Days 9-10: Polish and demo prep - End-to-end test: search a handle, view its profile, view its graph, run the infra scanner live against your test hidden service, export results. - Write the report/methodology document – judges for this problem statement care specifically about your approach to legal/ethical data collection, so document your synthetic-data design decision explicitly and well. - Prepare a walkthrough script: search -> find actor -> show graph connections -> show stylometric match to a rebranded persona -> run live infra scan -> export.

This gives you a working, demoable system with clear division of labor across all five team members, none of it dependent on accessing real dark web infrastructure.